Internet: network of network connecting a large number of endhosts via links or routers/switches. Technologies differ in the way, speed, reliability and range info is transmitted

Packet Switching

Organises how data is transmitted and how resources are allocated in the internet. Principle of splitting information into individual packets and routing these individually. Information is transmitted in units of **packets** of limited size that have **payload** and **address fields**. Access networks → customer facing, end hosts are attached to. Core networks → machines interconnected with each other through links that can carry large amounts of data.

Transmission Media How we actually carry data. Data transmission is based on the propagation of signals/waves over a transmission medium. **Acoustic signals** propagate through air or water at the speed of sound, represented as a variation of pressure over time. **Electromagnetic signals** propagate through space at the speed of light, represented as variations of amplitude, phase, or polarity over time. The medium distorts and attenuates the signals, the receiver needs to guess the information, called demodulation.

Guided Transmission Media A signal/data propagate in a confined space/cable. Twisted-pair cabling → two stations, susceptible to interference, cabling is easy e.g bends around corners. Used in telephone cabling and some Ethernet variants. Coaxial cabling → less flexible, multiple stations so many stations can send data to each other. Used in cable television and cable-based internet access. Better shielding from interference. Fiber cables → Backbone of internet nowadays. Long ranges (think undersea cables), immune to interference (fewer bit error rates), difficult to bend, two hosts.

Unguided Transmission Media Wireless transmission so you can be mobile. Signals can propagate in all directions, do this through radio frequencies e.g Terrestrial radio channel: Can propagate through walls (save on expensive cabling!). Satellite radio channel: link two far away ground stations without cabling, or for television and radio signals. Expensive!

Quality of Service - QoS and **QoE** (quality of experience) requirements of applications

QoS metrics characterize the performance of a network independently of applications. 3 typical measures in the internet: **End-to-end delay,** Packet loss rate, Achievable data rate/throughput. And for telephone networks: Call setup time → time that passes from you dialing to the first ring on the receivers end, Call blocking probability → probability of picking up phone and getting no signal.**QoE** measures are application dependent e.g think in zoom its perception-oriented performance such as speech quality

Delay End-to-end delay is the time between the generation of a packet at a source and its reception at the destination. It is made up of many different components listed below.

Round-trip delay is the time between the generation of a packet at a source and the reception of some response at the source.

Propagation delay → travel time for the signal to physically travel, d / v. Transmission delay → time to put all the packet's bits onto the link. Data rate R related to modulation, packet length (L), therefore l / R. Processing delay (in hosts and routers) → the time a router/host takes to examine and process the packet, i.e check header. Number of routers along a path. Queuing delay (in hosts and routers) → (drop-tailing behaviour if the queue is full)

Measures for Quality of Network

Bit Error Rate/Probability → Rate at which the receiver guesses bits wrongly due to distortions. We assume this is fixed probability. For example: probability all bits are correct → $(1 - p)^L$ for packet length L. For example: probability at least one is incorrect = $1 - (1 - p)^L$.

We use checksums to detect bit error rates, the receiver may decide to throw the packet away or e.g repair it if it is an image

Packet Loss Rate → Packets can be lost for many reasons, e.g end hosts crash, router buffer space is full, routers crash, protocols have internal errors, etc. Throughput measures the amount of data received per unit time (we choose this)

Communication Patterns (for Networked Applications)

Unicast → Exactly two pairs talk to each other

Broadcast → One node is a sender, all others are receivers

Multicast → One node is a sender, several but not all others are receivers. Join multicast group they join by invitation or subscription.

Paradigms for Networked Applications Different ways that applications communicate with each other over a network.

Client-Server Paradigm: Client: approaches with a request, the server may give a response back. The client must know the address of the server. Clients don't talk to each other. Server: group of computers reachable under well-known address, offers a service, always sitting active/waiting for client request. Must serve customers in parallel and be reliable (relying on one server creates a single point of failure). Peer-to-Peer Paradigm: A participant both receives AND transmits data. Efficient way of transferring larger files to a large number of clients For example, 3rd client finds out about the server and the first 2 clients, gets the first ⅓ from the server, etc

Circuit-Switching → Think telephone calls (guarantee) We can support **CBR traffic** (and VBR) through a dedicated circuit or connection between end points for the duration of the call → virtual private cable. Connection setup → identify route, set aside resources for this guarantee. Connection usage. Connection teardown → free the reserved resources. No resource re-use. When we get the call accept response, the second switch (one next to destination) will commit the resources to the call.

Packet-Switching → The required data rate is not the same all the time. We can support **VBR traffic**. Data flows are segmented into packets as they can be transmitted independently and individually. No resource reservation, reuse resources, no guarantee about delivery, packets may arrive unordered, require header (trailer) fields, routers in packet-switched networks perform more complex processing during transmission than switches in CS networks. How we deal with congestion/router buffer space being full: Drop the packet, let the source know they're sending data too fast, give packets different values (think gold, silver, bronze customer). No connection setup or teardown phase, cannot start sending a packet until it has been received completely, (needs checksum and destination, etc), forwarding /receiving can be parallel (pipelining)

Protocols and Services

Packets in internet get lost, delayed, reordered, etc. Network **protocols** are rulebooks that tell hosts how to deal with the various situations that can emerge (i.e recognise and repair). Different stations need to cooperate to run those procedures. **Services** are what a protocol provides. The TCP and IP protocol together allow you to achieve the service of reliable and in sequence delivery through the socket interface (API).

IP (Internet Protocol). The foundational protocol of the internet and we use version 4 (IPv4). For example the IP service compares well to the postcard. Defines what a packet is, what the header must contain, and there is no guarantee of delivery. It is a **best effort** service: Connectionless (no resources reserved), Unacknowledged, Unreliable, Unordered. IP Addresses are assigned to a network adapter (component that allows a device to connect to a network and send/receive data) in a host. They carry info about where in the internet the host holding that address sits. Humans normally work with human-readable host names e.g www.catnerbury.ac.nz. A protocol called domain name service (DNS) does the translation between the two

They are written in dotted decimal notation (32 bits wide, each byte separated by a decimal). Supposed to be worldwide unique

UDP and TCP. Two important protocols that work together with / 'on top' of IP. They ensure reliable delivery through port numbers. Think of IPv4 address = house/endhost. Port number = person within the house/particular application running in that end host. UDP/TCP packets are treated as payload of IP packets. Both are capable of setting up sessions/connections between two hosts

UDP → user datagram protocol. Offers the best effort service like IPv4 but adds port numbers. TCP → transmission control protocol. Allows for safe, reliable, ordered data transfer in the internet. Connection oriented so requires a TCP connection and teardown. Only exists in the two involved end hosts. So end hosts must agree on certain values i.e shared-state before starting. Reliable and in sequence, byte-stream oriented, full-duplex (transmission and reception can happen in both directions). Port numbers are local to the host and used to address different applications/processes/threads. 16 bit long unsigned integer. A process on a host can allocate one or more port numbers. One port number is at most allocated to one application. For example, typically port number 80 = HTTP/WWW (runs over TCP), 53 = DNS, etc

Processes and Blocking. A process is a running instance of a program / application. OS gives illusion that we have a single CPU for your program and a certain amount of memory allocated exclusively for you. In reality we have time-sharing. Processes can be in one of several states, Running, Runnable, Blocked → process is waiting for input, while block it does not use any CPU cycles, the OS will notice when input is received and wake up that process (better than constantly looping and checking if data has arrived!). Some socket calls (e.g accept() or select()) can be blocking

Sockets. Socket API makes sockets work similarly to files. You open, read/write, close, etc. A socket is bound to exactly one port number, Associated with the service of an underlying protocol (TCP or UDP), Associated with buffers which, Decouple the application from the underlying protocol. For example, when using send()/write() to send data within your application the system will put this data into a buffer belonging to the socket, then the underlying protocol will take the data out of this buffer and send, you have no control over when this happens (up to the application). Similarly, when send()/write() returns, this means the data has entered the buffer but not that it has been reliably received. Client/Server Paradigm in the UDP/TCP/IP Context → Client applications (on the internet with either TCP or UDP). Run on an end host. You have to know the IP address of that server, and the port number you want to reach on that server. If you don;t know the IP address of the host you can always use the localhost address. Open a socket, connect to the IP address and port number, send and receive data with the server, then close the socket down. Server applications Run on an end host. Open a socket, bind it to the IP address and port number that's advertised to clients, wait for incoming connection requests from clients, respond to them. Port numbers distinguish applications.

Socket Types. Stream sockets → offer a byte-stream service. Reliable and in sequence delivery of a data stream (often used with TCP). If delivery fails, the sending application is notified. Does not preserve record boundaries

Datagram sockets → offer a datagram service. Best effort i.e not guaranteed, nor reliable, nor acknowledge (often used with UDP). Offers delivery of distinct finite-sized datagrams / packets. Preserve record boundaries (receiver knows where message ends and next begins). Also raw sockets and sequenced-packet sockets

Socket API. TCP Client - Create a socket which allocates resources (buffer, socket handle), and assigns a random unused port number, Establish a connection with the server, Reading from / writing to socket. Close the connection

TCP Server. Create a socket. Bind it to a particular port number and IP address. Declare willingness to accept incoming connections, allocate resources (queue) for incoming connection requests (TCP). Accept an incoming connection request (i.e take it from the queue) and create a new socket for this connection, bound on the same port as the we called listen() on. Send and receive data in this new socket. Close this new socket, and at the end when our server software shutdowns we close the original socket. **UDP Client**. No need for connect(). Create a socket. Send and receive as much as you want, the receiver address must be supplied for sendto(), and recvfrom() returns the address (IP address and port number) of the transmitter and the actual data received. Close the socket. You can use connect() to supply a default receiver address.

UDP Server. Create a socket. Bind the server to an IP address. Then send and receive like before. Close the socket

Difference with CP is there are no listen() and no accept() calls required.

Main Socket API Functions. socket() → constructor that returns a socket object

```
from socket import socket

socket(family=AF_INET, type=SOCK_STREAM, proto=0, fileno=None)
```

Allocates resources like socket buffer, assigns random unused port number. family → AF_INET for IPv4, default. type → SOCK_STREAM for TCP, SOCK_DGRAM for UDP. (Optional) protocol → choose the protocol you want within the existing address family (0 means system gives best option, e.g for stream socket this is TCP). OSError.

connect() → request establishment of a connection (TCP / SOCK_STREAM) or specify a default receiver (UDP / SOCK_DGRAM) for send() and recv(). **s.connect(address)**. Address (for AF_INET) is a tuple of elements → (address_ipv4, port)/ Address and port number of server/intended receiver. The address can be supplied in dotted decimal or as a string e.g www.canterbury.ac.nz. socket.gaierror for address error, OSError, TimeoutError

send() methods → used to send data form ourselves to peer host. **amount = s.send(bytes, flags, address)** (either/both). Supply the data in the first argument as a bytes or bytearray object. send() requires a connect call made prior (so the destination address is known). sendto() requires the destination address to be supplied as a tuple like with connect(). Returns the actual number of bytes that have been placed in the socket send buffer. Raises the same errors as connect. If you send too much data for the socket buffer, the OS will block you until there is enough space, after which it will unblock you and return

recv() methods → used to read received data from a socket. bufsize → caller species buffer length (max amount of data that can be read by this call). flags → these functions are normally blocking, flags can change that (same for send()) s.settimeout() e.g for 5 seconds, if no data received behave accordingly. recv() → returns the actual data that can been received. recvfrom() → also returns the address tuple from the station you received from. OSError, TimeoutError.

```
data = s.recv(bufsize, flags=0)
data, address = s.recvfrom(bufsize, flags=0)
```

bind() → binds a socket to a particular IP address and/or port number. **s.bind(address)**. address is the address tuple we bind to. This is non-blocking (doesn't put you to sleep but can fail you). OSError. **listen()** → declares willingness to accept incoming stream connections (TCP) of type SOCK_STREAM or SOCK_SEQPAKCET. **s.listen(backlog=...)**. Allocates resources like queue for incoming connections, any excess connection request is denied. Only used on the server side. Also non-blocking. backlog → specify how long the queue is (Python has a default value for this). OSError

accept() → waits for incoming connection requests of stream type (TCP). **conn, address = s.accept()**. Returns a newly created socket with the same port number as socket previously created to access the new connection (conn) and gives you the address tuple of the peer host that sent the connection setup request (address). This is only used on a stream/TCP server. Normally blocking (i.e calling accept on empty queue - you will be blocked). OSError

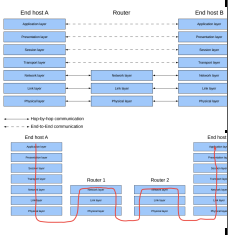
close() → call when you're done with a socket as it returns the resources (buffer space) to the OS for use by other applications. May be blocking (if you close a TCP socket where the underlying connection is not yet finished sending out data previously sent/buffered)

byte and bytearray objects - bytes are immutable, returned by recv() / recvfrom() and str.encode() which returns a string encoded as a bytes object. bytearrays are mutable, useful for assembling packets as we can update values by indexing/slicing

select() → can prevent busy-looping and wait for input from multiple sockets at once.. You provide a list of sockets to wait on, it blocks you until one has data, tells you which one this is.. Rather than calling recv() on the first one and getting blocked for all until that one has data. Endians for agreed network byte order. getaddrinfo() - Provides info about what services a host supports to query port numbers and protocols that are supported by a host. Service = combination of family, type, and protocol which can be used to create a socket able to connect to the host. Often necessary to determine how to connect to a host. Can also use the DNS to return an IP address given a textual domain name. family, type, and/or protocol → will filter the returned services. . Returns a list of 5-tuples for each service

OSI Reference Model

OSI = Open Systems Interconnection. First structuring of network software. We now use the TCP/IP model



Network layers use hop-by-hop communication to achieve end-to-end communication.

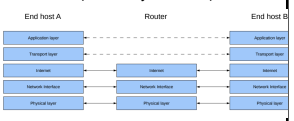
Lowest layers have single-hop scope and exchange PDUs only between physically connected hosts.

Upper four layers exchange PDUs between end hosts over routers and have end-to-end scope

Physical Layer → concerned with the transmission of digital data over a physical medium. Link Layer → aims to achieve reliable transfer of frames over a single link. Involves specification of Framing → how do packets look, their format, how to specify the start/end of a packet (delineation - bit pattern). Error control → how do we detect errors, what type of checksums do we use, how do we retransmit or use error-correction coding (adding redundant data). Medium access control → how do you distribute the right to talk, maintaining fairness. Flow control → receiver cannot process data at the rate the sender sends it. Network Layer → concerned with providing a link technology-independent abstraction of an entire network to higher layers (same as in TCP/IP). e.g don't care about journey our letter takes. Now packets. Also concerned with addressing (need an addressing scheme that covers entire network) and it must be capable of end-to-end delivery of packets (this layer helps routers make routing decisions by giving it info for this) Transport Layer → concerned with reliable, in-sequence and transparent end-to-end data transfer (TCP sockets!) and should offer suitable programming abstractions (interface) to higher layers. Involves the specification of Error control → if the link layer does this along every hop why do we need this? The link layer error control protects you against errors during packet transmission on a link, once the packet reaches the router, the protection is off. Routers may drop pacers (crash, full buffers), hence we take care of this here. Flow control → end-to-end procedures because you want to slow down for your receiving end hosts (vs the next hop you take to reach the end host). Congestion control → allow you to slow down the rate of transmission if some parts of the network become congested (i.e buffers/queues at those parts of the network are so full they are throwing away packets). Session Layer → giving end hosts the opportunity to organise communication sessions with more structure. A session can involve several transport layer connections in parallel or sequentially. A session might control the way two end hosts interact, for example, enforce that they speak alternatingly. Representation Layer → the ability to translate different data representations with different hosts. For example, the issue with big- and little-endian. Application Layer → can be the application itself or common services the application needs

For example File transfer services, Directory services → for example, allows you to maintain a central database of users and their authority, Transaction processing support → doing transactions in an assured and reliable way even in the presence of errors (so money isn't lost!)

TCP/IP Reference Model



Used in the internet, we aim to keep layers 1-3 simple, so routers can be simple for performance.

Lowest 4 layers are essentially the same as OSI. Application, Presentation and Session layer lumped into Application layer (i.e you need to take care yourself of data representation or how to structure your sessions)

Application Layer → applications/useful for applications. Uses services of the transport layer through socket interfaces (sockets). Transport Layer → provides end-to-end delivery of data/communications to applications, and offers these services through socket interface. Its SAPs are sockets used for application multiplexing (through port #'s). We know the protocols TCP and UDP. So several applications can use transport service, a port is bound to one application, ports are identified by numbers, the PDUs generated by TCP/UDP are called segments, these include the port number, the TCP/UDP receiver delivers incoming segments to the application denoted by the port number (through an associated socket) TCP has mechanisms for Error control (retransmission-based) and in-order delivery, Flow control, Congestion control. UDP has none of the above. For transmission TCP and UDP hand over segments [as payload] TO the [underlying IP protocol on the] internet layer. For reception TCP and UDP get incoming segments FROM the internet layer. Internet Layer → the central layer in the TCP/IP model, it uses IP (internet protocol) which Specifies an addressing scheme (IP addresses), Provides end-to-end delivery of datagrams (i.e packets), Does not specify how routing is done (this is left to dedicate protocols), Has no mechanisms for error-, -flow-, and congestion control Hence it is so simple and central! Can send IP datagrams over any network interface (e.g Ethernet, optical, carrier pigeons, etc) as things are so simple. There are no further dependencies between the IP layer and the underlying technologies (i.e below the bottleneck, they just take the IP packet and send it off)

Physical Layer → similarly to the PHY in OSI RM

Network Interface Layer → similar to the link layer in the OSI RM. Need to accept IP datagrams and deliver them over a physical link. Need to receive IP datagrams and deliver them to local IP layer. They do things like medium access control, framing, address resolution and MAY also include link-layer error- and flow control

Service Primitives

How does a service provider (layer N) and service user (layer N+1) actually **interact** with each other? They talk through service primitives. They have to obey rules that may govern in which order service primitives can be used (enforced by interfaces). We have s.request, s.indication, s.response, s.confirmation. Can be implemented in different ways in a programming language or library or OS.Some typical interactions on a service interface are: Confirmed Service s.request from user to provider, who makes packets, sends to B provider, s.indication to inform B user, who sends s.response back, A provider sends s.confirm to user.

Unconfirmed Service Just sending process of above. Service user A knows nothing about whether data made it or what B user thought.

Confirmed Delivery Service Service provider B sends ack to provider A who s.confirms with user. Confirmation your postcard made it to the house, but it could have gone to anyone in the house.

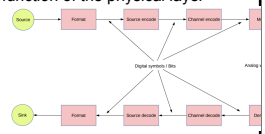
Some Standard Protocol Mechanisms

Multiplexing → combines multiple separate data streams so they can share the same communication channel (transport data from several layer N SAPs over 1 layer N-1 SAP), N protocol entity needs to make scheduling decision of which SAP to serve next and differentiate SAPs and include a SAP identifier into the N PDU to allow receiver N entity to delivery an incoming N-PDU to the right SAP. TCP uses port numbers for this.

Splitting → N layer protocol entity can transmit any data it received from a layer N SAP over layer several layer N-1 SAPs. Allows transmission of data over several channels to increase throughput or reliability through parallel transmission. N-entity needs to make scheduling decisions on which N-1 SAPs to use for a given PDU.

Fragmentation and Reassembly → for different MTUs. Incur extra overhead, frags must be numbers, receiver sets up buffer (and timer if no ack e.g IP). Blocking and Deblocking → block if small N-SDUs, set up buffer size and timer to transmit, save overhead. Sequence Numbers → maintain local counter to detect gaps, reorder correctly, find duplicates - issue of wrapping.

Digital data → bits, bytes, sequence of discrete symbols, symbol (0 and 1) = member of a finite set known as alphabet, we want receiver to recover bit values perfectly without errors (reduce probability of error, allow perfect recovery), but they're put onto analog signals (distorted).
Analog data → can take uncountably many different values e.g voltages. Cannot allow perfect signal reconstruction.
So modulation is a key function of the physical layer
View of physical layer → shows all transformations data to be transmitted goes through



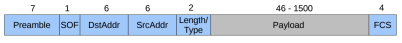
Formatting → turns source data into digital data (e.g analog to digital). Output is a stream of bits.Source coding → compressing data that is too large. Lossless compression = digital data is encoded to be perfectly restored, good for text, webpages, software. Lossy compression = data after decompression may not be the same. Give up quality based on human perception limitations and have better compression ratios.
Channel coding → enlarge data by adding redundant symbols/bits to counter transmission errors i.e reduce bit error probability. Separate source and channel coding as we are still optimal this way, therefore easier to work with.
Modulator → maps sequence of digital symbols/bits to a physical signal or waveform of a certain duration (symbol time) suitable for transmission over the channel. 0-bit is mapped to some waveform $s_0(t)$, 1-bit to $s_1(t)$. Channel → guides the signal/waveform from transmitter to receiver.
Demodulator → processes incoming distorted waveforms and guesses what was transmitted (possible with finite number of waveforms $s_0(t)$, $s_1(t)$). Probability of error or symbol/bit error rate = chance guess is wrong. Channel decoder → takes output from demodulator, tries to use the redundant bits to correct errors. Source decoder → uncompressed channel decoder output. Formatter → shows output to user, often involves digital to analog conversion
Baseband Transmission. Non-Return-to-Zero (NRZ) Encoding. Transmits bits over a cable by changing voltages. Each bit is assigned a voltage we hold for the symbol time.
Bipolar NRZ Encoding → 1 = V, 0 = -V
Unipolar NRZ Encoding → 1 = V, 0 = 0V
M-ary Signaling → groups of k bits are mapped to 2^k different voltage levels, however harder to distinguish or require more power and increase symbol error rate.
Signal Impairments s(t) → actual output, r(t) → received output Attenuation → signal strength decays with distance. Measure → transmitted signal power / power of signal seen at receiver = η (eta) to characterize attenuation factor (can rearrange to calculate received power). Convert to decibels → $\eta dB = 10 \times \log_{10}\eta$
It can be frequency dependent. Through Fourier analysis a signal can be represented as a weighted sum of sine and cosine waves of different frequencies. A channel can act differently on different frequencies (think coloured glass only letting in some light) which changes what the receiver gets. e.g the channel might transmit the first few harmonics and cut off any further ones (approximations tend to get better with higher cutoff points). Frequencies are not cut off sharply, and the channel might attenuate different frequencies differently.
Thermal Noise → wide range of other impairments referred to as noise. Can be created in the channel or receiver circuitry. Thermal nose results from random movements of electrons in circuitry.
Passband Transmission
With baseband transmission the medium cannot be shared otherwise we have collisions. Passband transmission transmits data by modulating a sinusoidal carrier of a certain centre frequency. The resulting signal occupies a certain bandwidth around that centre, you must ensure sufficient separation of centre frequencies → known as frequency-division multiplexing as we can send in parallel.

General form of the transmitted signal in passband modulation:

$$s(t) = A(t) \cdot \cos[(f_c + f(t)) \cdot t + \phi(t)] \qquad , \qquad t \in [0,$$

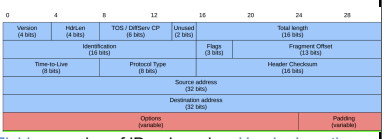
In digital passband modulation digital info is modulated onto a sinusoidal carrier of duration T (symbol duration). A(t) = amplitude, f(t) = frequency offset, ϕ(t) = additive phase, fc = centre frequency
3 pure forms of digital modulation with A(t), f(t), and ϕ(t) → can have combos of this
Amplitude Shift Keying → amplitude modulation (A(t))
Frequency Shift Keying → vary the frequency f(t)
Phase Shift Keying → vary the phase offset ϕ(t) (flip by π)
Synchronisation
Sender and receiver need to agree on symbol time, centre frequency, how to make start and end of packets, they do not have a common source of times and frequency. Each have their own crystal oscillators to generate a time reference, but often wrong (old, physical distortion). The receiver must estimate the sensor clock out of the transmitted signal called synchronization.
Symbol and Bit Synchronisation → how the receiver figures out senders signal time (for large streams, need changes often to avoid drifting apart)
Manchester Encoding → 1-bit = change from 1 to 0 (change in middle). 0-bit = signal change from 0 to 1. Never drift as we always have signal changes. Receiver will sample bit time at much higher resolutions to track clock of transmitter. Preamble → e.g in Ethernet, starting packet will well-known sequence for initial synchronization
Frame Synchronisation → receiver must know start and end of frame.For example. Use SYNC preamble to mark start of frame (where SYNC pattern ends and frame starts). To know where the preamble ends/PHY frame starts use SFD (start-frame-delimiter) OR Time gap (idle between frames), code violations length fields (checksum mustn't fail), start/end flags (sender must ensure this does not show up in data send)

LANs (think Ethernet, Wi-Fi) → packet-switched networks with limited geographical extension, controlled by one owner, offer high data rates, low cost to attach stations, shared transmission medium. Need to agree on how and when to use this medium (when can a station talk).LANs cover the physical layer (specifying transmission media, data rates, network topologies) and medium access control sublayer (MAC) (specifying how stations share the medium) - maybe also link layer with error and flow control.
MAC bridging and Logical Link Control (e.g the error/flow control) are independent of individual technologies such as Ethernet.
Physical Topologies → what does the cabling look like, what properties do we get.
Bus Topology → Stations are attached via tap line, bus is a broadcast medium, parallel transmission collide, frames still have intended receiver (dist addr, not force others to pick it up, reduce workload), limited bus length to limit signal attenuations → we want to guarantee transmission is heard by all. Star Topology → a central node with each station attached through their own private cable (one for trans, one for recv). Central unit can be a Repeater → bus behavior, Switch/Bridge → transmission is only heard by intended receiver. Wireless Topologies → complicated, no guarantees about all stations hearing, trans is time-varying, links influenced by propagation environment
MAC Fundamentals
MAC protocols allow a number of stations to share a common medium, influenced by properties of the transmission medium. Assume broadcast medium (heard by all), packets are destroyed in parallel transmission for all. We want: Small medium access delay (time between arrival of frame to empty station from higher layers and start of successful transmission), fairness, fair-reuse of unused resources, efficiency (low overhead, high throughput), stability (↑ load should not ↓ throughput)
(Orthogonal) MAC Schemes → i.e transmission / behaviour of one station has no impact on any other station, guarantee collision-free transmission.
FDMA → Frequency Division Multiple Access. Good for CBR, bad for VBR. Split along frequency access. Only the owning station can transmit on allocated sub-channel. To receive either make separate receive on each sub-channel (expensive!) or tunable receiver (receiver operates in only one sub-channel, tune it to switch → costs time, requires coordination!) Completion Time → total bandwidth = B, station is assigned 1/N of it for fair share, so medium access delay if the station is empty is zero (can start transmission immediately). If the packet has size B/N bits, once transmission immediately starts, it takes 1 second to transmit that packet (B/N length / B/N data rate on sub-channel = 1). Therefore total completion time = MA delay + transmission delay = 0 + 1. Advantages → Transmit in parallel, no need to time synchro between transmitters. Disadvantages → need N receiver or tunable receiver, frequency synchro required, no re-use, coordination and shared state for allocating sub-channels e.g which frequency I use.
TDMA → Time Division Multiple Access Good for CBR, bad for VBR
Subdivide time into superframes. Each superframe will have timeslots (from 1 to N). During one slot station can use the entire frequency band.Completion Time → forget guard times, each station gets the full B bandwidth (bits per second) for a fraction of 1/N time. Assumed superframe length is 1 second (TSF), that time slot is sufficient to transmit B/N bits. For medium access delay we cannot start sending immediately, assume TSF / 2 for average as packet arrival is random = 0.5 seconds. Transmission delay is 1/N seconds as entire bandwidth (we get to 1/N seconds) is sufficient. As long as N > 2, this is quicker than FDMA → start later, finish earlier. Advantages → easier to give different stations different bandwidth by giving multiple time slots (in FDMA one station needs 3 different units of transmit circuitry which is harder to set up), better completion times, no tunable receivers. Disadvantages → tight time synchro, high expected access delay (even in idle stations), not possible to re-use unused time slots, coordination and shared state required for allocating time slots.
Random Access Protocols
Simpler than orthogonal protocols. Don't reserve resources, no central station, low implementation complexity. Risk of collisions to save coordination overhead and improve overall efficiency.
ALOHA → MAC protocol. We assume N uncoordinated transmitters, one receiver/base station, collisions occur and are bad!
Pure ALOHA → Frame is transmitted immediately (no coordination with other stations), acknowledgment timer is started. The receiver sends an immediate ack if successful reception (timer cancelled), otherwise transmitter assumes failures (possibly due to collision). When the ack timer expires, enter backoff mode (random waiting time) then re-transmit. Randomly avoid collisions again. After a number of attempts, give up and drop the frame.
Waiting times may impact throughput (if chosen too small for lots of contenders or too high for fewer contenders). Advantages → very simple, best protocol for small network load, medium access delay is 0, can use whole bandwidth while transmitting, probability of collision is lower. Disadvantages → problems arise if network load increases, this increases rate of collision (when B starts its frame during the two frame times longs vulnerability period of A's frame, they collide)
CSMA Family of Protocols → Carrier Sense Multiple Access
Stations can listen on the channel → tell when it's idle or busy (ability is called carrier-sensing)
Nonpersistent CSMA → if busy, draw random waiting time, sense the medium again, repeat. If idle, transmit. If collision (voltage > voltage you put on cable), wait random backoff time. Channel may be underutilised if changes from busy to idle but we're waiting random time.
p-persistent CSMA → if busy, listen for idle, divide time on idle medium into small time slots (large enough to carrier-sense and detect if another station starts transmission), for each time slot, perform random experiment probability p it transmits. After each experiment, perform another carrier-sense, if still idle, repeat experiment, otherwise restart. Many or few contenders will determine a good p.
CSMA/CD or 1-persistent CSMA → CSMA with collision detection. If idle, transmit. If busy, wait for idle and transmit immediately. Good throughput/utilisation if on one else listening, otherwise collisions. CSMA/CD should detect collisions while still transmitting (e.g voltages), then abort transmission, do random backoff, attempt again.
Ethernet →Packet-switched LAN technology.

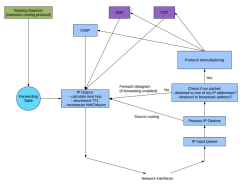


SOF → start of frame/preamble ends, FCS → type of checksum
0-bit padding added to get minimum length (higher layers e.g IP have length fields to help receiver distinguish this)
Ethernet Addresses → 48-bits, globally unique, vendors get Ethernet ranges and assign addresses to individual network adapters. 6-bytes in hexadecimal separated by colons. Broadcast address FF:FF:... to go to every station in this Ethernet, otherwise multicast addresses (refer to a multicast group) have least significant bit on leftmost byte 1, 0 for unicast.
Length/Type Field → for values > 6000 hex, type field (tells which higher layer protocol generated the packet in the payload → for protocol multiplexing), ≤ 1500 first two bytes of payload are type field, > 1500 no field indicating frame length (infer from PHY layer).
Ethernet PHYs → several different PHY layers (guided trans examples).
Ethernet supports 2 types of topologies → Broadcast topologies: coaxial cable, MAC needed as stations share capacity of medium. Half-duplex Ethernet (cannot send and receive in parallel, this uses CSMA/CD). Switched Topologies: stations attached to switch with a private link (2, Full-duplex operation). No MAC required, competition moved to switch rather than medium.
Half-duplex Ethernet uses CSMA/CD. New frame arrives at MAC of Ethernet station, set local counter to 0, when collision detected abort frame transmission with jamming signal and increase counter, if counter > 16, drop frame and reset or wait backoff time. We calculate backoff time as [0, 1, .. 2^**min**{10, count} -1] doubles to avoid issue of too many or few contenders (interval called backoff window) multiplied by a slot time (just large enough to cover max round-trip time and processing delays for collision detection). Very short medium access delay under light load.
Minimum Frame Size → ensure a transmitter has the chance to detect a collision while still transmitting.
Bridges, Switches Bridges connect LANs at the MAC (Layer 2) level. They inspect MAC source/ destination addresses, learn which LAN a device is on from source addresses, and forward frames only when necessary; they can improve performance by allowing parallel transmissions and prevent faulty signals from spreading between LANs. Bridges use a loop-prevention protocol to detect loops and disable certain links. Layer-2 switches are essentially centralised bridges where each station has its own full-duplex link to the switch, eliminating channel competition/MAC contention. The switch forwards frames to the correct output in parallel when destinations differ; if multiple frames target the same destination, it buffers them and sends them sequentially. If the buffer fills, additional frames are dropped.

End-to-end internet principle → routers should be kept simple and only be related to packet forwarding and minimal service guarantee (i.e. no guarantee). Leave everything else to higher layers/protocols (think TCP ensures reliability)
IPv4
Network layer protocol of the internet. IP addresses are assigned to the network interfaces rather than hosts (e.g each Ethernet adapter in a host has an IP address), however they often use one at a time (THE IP address). Basic IP service is datagram delivery which is connectionless, unacknowledged, unreliable, unordered → **best effort**.



Packet Format
32-bit or 64-bit computer → size of data bus between processor and memory. Version Field → version of IP we're using. Header Length → in multiples of 4 bytes, e.g with no options, this is 5 as 5 x (4 x 8) = 5 x 32. TOS → often ignored, used to achieve QoS e.g expression priorities. Length → need in Ethernet. Identification → sender of packets marks each packet with new number (good for fragmentation and packet loss).
Flags → e.g DF (don't fragment), MF (more fragments coming) Offset → starting index of present fragment within the entire message.
Time-to-Live → how many routers a packet can visit before it is dropped. Protocol Type → tells us about higher layer payload stuffed into data to provide protocol multiplexing/SAPs. Header Checksum → for header, not payload (e.g in video payload errors aren't a problem, and checksum over entire packet is costly, especially when routers recalculate checksum as they decrement time to live).
IP Addressing → have internal structure, they're partitioned into two parts. Network identifier <network-id> denotes a network e.g Ethernet. Host identifier <host-id> identifies one particular host/network adapter within that network. Must be unique for that network.
So these carry topological information (tell you where in the network you are, like name of recipient and street information on a letter). If you change your network your IP changes, however IP does not support mobility (ongoing connections break!)
CIDR → Classless Inter-Domain Routing
Makes the boundary between the network and host part flexible as e.g we may not know how many bits to allocate network id. To specify an IP network you must specify the IP address of that network, and the network mask value (netmask) → how many bits of address are from network part, how many are from host part. Netmask → left-most k bits are 1, the rest are 0. Bitwise AND this with the host address to get the hosts network address. Host address with all 0's = referring to entire network. Host address with all 1's = broadcast address of this network. All other addresses can be assigned to individual hosts. E.g 192.168.40.64/28 has (16 - 2) 14 host addresses, the value 0000 is part of the network-id, 1111 is the broadcast address. There are also reserved IP ranges (i.e /k value). Private IP networks can be used internally but must not be shared into the internet (routers throw these away).
IP Forwarding and Routing



Receiving (RHS). IP packet received within Ethernet frame. Checked for correctness and sorted in IP input queue (i.e correct IP header checksum, right value in IP version field). Packet options are processed (rarely used). Check if the packet is destined to this device (directly or through broadcast), if so protocol demultiplexing is carried out (look at protocol field in IP header, check e.g if payload is TCP or UDP, hand over to correct protocol), if not, if packet forwarding is not enabled (i.e are we a router) drop it, if so put it in the IP Output stage.
IP Output Stage (centre). Consult forwarding table to determine next hop/outgoing interface (if this cannot be determined, drop the packet). Decrement TTL, drop packet if this reaches 0. Recompute packet header checksum, hand packet over to outgoing interface.
Forwarding table → maintained by a routing daemon (process executing a routing protocol) that fills the table with destination IP addresses (usually a network address with netmask), information about next hop (either IP address of next-hop or IP address of directly connected network), specification of outgoing interface (e.g Ethernet 0 vs 3) . ip route to inspect forwarding table.
The view routers have on the network topology is limited (only care about the next hop). If our forwarding table does not contain an entry that matches the destination, the table has a special default entry, so we send packets off to the default router who figures out where to send it for us.
Address matching → how we match a dist address with a table entry. dst AND /k netmask == a.b.c.d AND /k netmask. If a default route is added to the forwarding table, any time this check fails that packet is handed off to the default router.
Forwarding tables in hosts → This way forwarding tables in end hosts only need a few entries (one for each network is directly attached to, and the default route/routers so your end host doesn't have to participate in a routing protocol to learn about all the destinations in the world individually).
Forwarding tables in routers → fringe routers (close to network edge) only have forwarding table entries for a subset of all networks attached to the internet, core routers are their default routers. Code routers maintain connection to the outside world, don't have default routers, must know almost all the internet networks.
Fragmentation and Reassembly (overall not good)
IP can run on various technologies, each of these have their own limitations of frame length called maximum transmission unit (MTU). Higher layer protocols like TCP and UDP shouldn't need to know this (separation of concerns, paths change over time). To deal with this, IP offers a max length of 65,535 bytes (including header) to higher layers, hacks payload down into smaller fragments, transmits them individually, uses header for reassembly. Routers can hack fragments down further if too big to fit over the next hop. DF forbids this, sends ICMP back (provide source node info about workable packet sizes). Routers should not reassemble (no guarantee a router gets all the packets, may have to fragment them again).
The receiver creates a buffer to receive all packets and starts a timer. If we receive all packets before the timer is up, deallocate buffer, reassemble payload and hand it to TCP or UDP. If not, the receiver drops what they have, buffer is freed, receiver may send back an ICMP that timer expired. There is no info about how large to make the buffer,, so receiver makes it full 65,535 → people may eat this up!